

xinu-rpi4 — OS としての改良余地アセスメント

Raspberry Pi 4 (BCM2711 / Cortex-A72, AArch64) 向け Embedded Xinu

2026 年 7 月 20 日

対象	Pi 4 カーネル (compile/make pi4 → kernel8.img)
基準コミット	ccb6437 (proc: harden tickless one-shot against total hang)
規模	.c/.h/.S 130 ファイル、約 31.4k 行、195 コミット (extern/ obj/ references/ を除く)

本書は Pi 4 カーネルを「OS として」評価し、改良余地を影響度と工数で順位付けしたものである。スケジューラ／メモリ／ネットワーク・ドライバ／ファイルシステムとツールリングの 4 領域を精読した結果をまとめている。重大な所見は実際にソースで裏取りし、そのうち Tier 0 の 5 件と Tier 1 の 4 件は本アセスメントと同日に修正・実機検証した (第 5 章・第 6 章)。

行番号はアセスメント時点のもので、以後の編集でずれる。関数名を併記しているのでそちらを頼りにすること。Markdown 版は docs/OS_ASSESSMENT_JA.md。

目次

1	総評	1
2	Tier 0 — 実害があり修正コストが小さいもの	2
2.1	ヒープ分割の 8 バイト境界オーバーラン — mem/memory.c getmem()	2
2.2	ヒープが IRQ / プリエンプション非安全 — mem/memory.c	2
2.3	ICMP エコー応答からのメモリ情報漏洩 — system/net_responder.c:210-234	2
2.4	SYN 4 発で恒久 DoS — system/tcp_server.c alloc_conn()	3
2.5	cat /microsd/xxx が物理アドレス 0 を読む — shell/fscmd.c:127	3
2.6	W^X が壊れている — system/mmu.c:191,214	3
3	Tier 1 — 設計上の本丸	3
3.1	優先度プリエンプションが実質ラウンドロビン — system/proc.c proc_resched()	3
3.2	(8) ready リストが O(n)、タイマ ISR が毎回 proctab を 2 周	3
3.3	(9) タイムベースが壊れている	4
3.4	(10) D-cache OFF が最大の性能レバー、ただし解放には DMA 規律が要る	4

3.5	(11) セマフォが偽物、network/ が丸ごとデッドコード	4
3.6	(12) スタックガードが皆無	5
3.7	(13) 2 GB のうち 1 GB が使えない	5
3.8	(14) TCP に再送も分割もない	5
3.9	(15) GENET TX が 50 ms のビジーループ	5
3.10	(16) DHCP が飾り	5
3.11	(17) ユーザ空間が存在しない	6
4	Tier 2 — ファイルシステムとエンジニアリング実践	6
4.1	(18) FAT32 にキャッシュがなく、共有 static が無防備	6
4.2	(19) VFS が抽象化になっていない	6
4.3	(20) 自動テストが実質ゼロ	6
4.4	(21) 重複と腐敗	7
5	推奨着手順序と進捗	7
6	第 1 次改良 — Tier 0 の 5 件	8
6.1	ホストテスト基盤の追加 — test/host/	8
6.2	実機検証結果	8
7	第 2 次改良 — Tier 1 の (7)(8)(12)(14)	9
7.1	スケジューラ	9
7.2	ネットワーク (14)	9
7.3	/chainload が箱を落としていた真因	10
7.4	決着した懸案 — /preempt は既存の壊れ	10
8	未解決の課題	10

1 総評

上半分 (プロセス / IRQ アーキテクチャ) は本物、下半分 (メモリ安全性・ネットワーク・ファイルシステム) はデモ級という非対称な成熟度にある。

評価できる点:

- **ISR の中で重い処理をしていない。** net プロセス + app ワーカー + aipl プロセスを park/kick で分離し (loader/main.c:206-269)、タイマ ISR はティック更新と RT スリーパの起床のみ、GENET ISR は計数・自己マスク・ack のみ。HTTP 処理は必ずプロセス文脈で走る。
- **フォルト後も生き延びる。** recover_spin() (system/exception.c:61) が AIPL ヒープロックを解放し IRQ を再開してスピンするので、同期例外のあとも HTTP が応答し /fault で原因を読める。
- **MMIO プローブが箱を殺さない。** safe_mmio_read32/write32 (system/exception.c:79-104) が __builtin_setjmp でガードし、ゲートされた PCIe レジスタへの書き込みが -1 を返すだけで済む。

- ヘッダコメントの質が高い。device/sd/sd.c:73-86 (74 クロックの電源投入ウィンドウ)、fs/fat32.c:180-184 (クラスタ確保の off-by-2) など、根本原因が文章として残っている。

いずれも「1 回のハングが電源再投入 1 回」という開発コストへの対処として設計が筋道立っている。

弱いのは、**落ちないので気づけない類のバグ**が各層に残っていること。以下はその棚卸しである。

2 Tier 0 — 実害があり修正コストが小さいもの

すべてソースで裏取り済み。本アセスメントと同日に全件修正した (第 5 章)。

2.1 ヒープ分割の 8 バイト境界オーバーラン — mem/memory.c getmem()

```
if (curr->mlength > nbytes) {           /* ← 残り 8 バイトでも成立 */
    struct memblk *leftover = (curr + nbytes);
    leftover->mnext = ...; leftover->mlength = ...; /* 16 バイト書く */
```

struct memblk は 16 バイト、ROUNDMB は 8 バイト粒度。残余がちょうど 8 バイトのとき、**返却したブロックの隣に 8 バイトはみ出して書き、偽の free ノードをリストに繋ぐ**。以後ヒープが静かに壊れる。

同型の欠陥が freemem() 側にもあった (8 バイト領域に 16 バイトのヘッダを書く)。こちらは目視では見落としており、ホスト側のランダム churn テストが検出した。

2.2 ヒープが IRQ / プリエンプション非安全 — mem/memory.c

getmem/freemem/kmalloc/kfree に critical section が一切ない。プリエンプションは実際に有効化される経路がある (/netpreempt, /rtos-jitter, preempt_demo)。proc_create は getmem を **irq_save** の外で、proc_kill は freemem を中で呼んでおり (system/proc.c:207 vs :422)、この非対称さ自体が症状である。

2.3 ICMP エコー応答からのメモリ情報漏洩 — system/net_responder.c:210-234

```
int total_len = ip[2]<<8 | ip[3];
if (icmp_len < 8 || total_len + 14 > 1518) return 0; /* len と比較していない */
int reply_len = 14 + total_len;
for (int i = 0; i < reply_len; i++) tx_reply[i] = frame[i];
```

total_len を実受信長 len と照合していない。60 バイトの ICMP に total_len=1400 を書いて送ると、RX リング上の隣接メモリ (=直前のパケット) 約 1.3 KB を攻撃者に送り返す。

同型の欠陥が system/tcp_server.c:2537-2541 (data_off も未検証。data がフレーム外を指し、偽の data_len が peer_seq を desync させる)。なお IP / TCP / UDP / ICMP のいずれもチェックサムを検証していない。

2.4 SYN 4 発で恒久 DoS — system/tcp_server.c alloc_conn()

struct tcp_conn にタイムスタンプがなく、状態は受信パケットでしか進まない。再送タイマもない。ACK を返さない SYN を NCONN(=4) 本打つと全スロットが SYN_RCVD で永久に固まり、再起動まで HTTP が死ぬ。alloc_conn() は LISTEN/CLOSED しか回収しない。

2.5 cat /microsd/xxx が物理アドレス 0 を読む — shell/fscmd.c:127

FAT32 のファイルを VFS に登録する際 node->size は入れるが node->data は NULL のまま (意図的、コメントあり)。cat/cp/mv はそれを無条件に deref する。アドレス 0 が RW 識別マップ (system/mmu.c:200) なのでフォルトせず、**低位 RAM の中身を表示・コピーする**。ls はもっともらしいサイズを出すので気づけない。

2.6 W^X が壊れている — system/mmu.c:191,214

kern_2mb = _start & ~2MiB = 0x0 なので、属性を分ける L3 (4 KiB ページ) 層が覆うのは 0x0--0x200000 だけ。実際の _data = 0x268000。つまり **.rodata の約 416 KB と .data/.bss 全部 (=全 DMA リング、proctab、smp_stack) が 2 MiB ブロックの RW+X になっている**。イメージが 2 MiB を超えた時点で静かに劣化したもので、ビルド時アサートもない。

3 Tier 1 — 設計上の本丸

3.1 優先度プリエンプションが実質ラウンドロビン — system/proc.c proc_resched()

proc_resched() が走行中プロセスを ready に戻す前に最良候補を pop する (:283 vs :292-295) ので、現在プロセスの優先度が比較に参加しない。prio=50 の RT タスクが prio=5 の hog に叩き落とされる。g_dbg_hi_dispatch カウンタ (:124) は、これが追われていた形跡である。

関連して、NULLPROC は決してプリエンプトされず (:403) ready リストにも載らない (:292)。NULLPROC はシェル／ウィンドウマネージャのループそのものなので、対話文脈は純粋に協調的で、net_yield_tick の明示 proc_yield() でしか進まない。さらに AIPL アクター実行中はプリエンプションが全面的に無効化される (proc.c:401)。つまり主要ワークロードでは、このカーネルは協調型である。

修正は数行。P1 (RTOS 化) の看板そのものなので優先度は高い。

3.2 (8) ready リストが O(n)、タイマ ISR が毎回 proctab を 2 周

- ready_pop() は最大 prio の線形探索 (system/proc.c:107-126)。NPROC=2048、N-Queens で約 1300 アクター、しかも **D-cache は意図的に OFF** (system/mmu.c:263) なので 1 ノード = 1 DRAM 往復。
- proc_next_delay_us() (:342-355) と proc_timer_tick() (:377-390) が同一 ISR で 2048 エントリを 2 周。最大 5 kHz。PROC_TICK_MIN_US のコメント (:337-341) 自身が、この ISR がシステムを餓死

させうると認めている。

→ 優先度別 FIFO + ビットマップ (CLZ) で O(1) 化、スリープはソート済みデルタキューへ。

3.3 (9) タイムベースが壊れている

timer_ticks() は可変長のワンショット発火回数であって時計ではない。にもかかわらず 4 つのサブシステムが固定周期として扱っている: actorproc.c:150,282,335 (* 10 = 100 Hz 前提)、cc/cc.c:514、device/usb/uspi_glue.c:95-100、device/usb/xhci/xhci.c:1119-1476。アクター GC の経過時間もキーリピートも約 10 倍ずれ、しかも非線形。CNTPCT_EL0 由来の真の now_ms() は既に 2 箇所にある。

3.4 (10) D-cache OFF が最大の性能レバー、ただし解放には DMA 規律が要る

SCTLR.C=0 は SMP コヒーレンスを「全アクセスが RAM に届く」で担保する設計判断 (include/smp.h:10-12) で理屈は通っているが、4 コア分の D-cache を捨てている。DCACHE_ON を有効にする前提として:

ドライバ	cache maintenance
mailbox (device/mbox/mbox.c:51-53,85-87)	正しい
video (device/video/video.c:247-249)	正しい
smp mailbox (system/smp.c:47-50,110)	正しい
JIT (cc/cc.c:723-735)	正しい (dc civac + ic iallu)
wifi (device/wifi/wifi.c:2138-2141)	部分的
genet (device/genet/genet.c)	dc 命令ゼロ (dsb sy のみ)
xhci (device/usb/xhci/xhci.c)	dc 命令ゼロ (dsb sy のみ)

規律は存在するのに 2 大 DMA ドライバだけ抜けている。DCACHE_ON は現状どのターゲットも定義しておらず、実質デッドコードである。

3.5 (11) セマフォが偽物、network/ が丸ごとデッドコード

system/xinu_compat.c:86-102 の wait()/signal() はカウンタの増減のみで、キューなし・ブロックなし。struct sement.queue は宣言されて未使用。sleep() は即 return、yield() は yield しない、create() はスレッドを作らない。

その結果、network/** (ARP/IPv4/ICMP/netaddr 約 1500 行) は完全にデッドコードである。netUp/netRecv/ipv4Recv/arpRecv の呼び出しは network/ の外にコメントとしてしか存在しない。それでも compile/Makefile:47-49 はビルドし続けている。生かす (本物の netif + ブロックするセマフォ) か消すかの決断が要る。

なお、ネットワークスタックは現在 4 系統が並存している: system/net_responder.c (ARP+ICMP)、system/dhcp_client.c、system/tcp_server.c、device/wifi/wifi.c:1238-2100 (ARP+ICMP+DHCP+DNS+NTF を独自に再実装)。

3.6 (12) スタックガードが皆無

カナリアも guard page も高水位マークもない。一方 IRQ エントリは **768 バイトのフレーム** (GPR 31 本 + q0-q31 全部、system/exception_vectors.S:97-131) を割り込まれたプロセスのスタックに積み、proc_create は 1024 バイトのスタックを受け付ける (proc.c:192)。NULLPROC に至っては sp = _start = 0x80000 で下方向に firmware spin table へ伸びる、長さ未知のスタックである。

3.7 (13) 2 GB のうち 1 GB が使えない

HEAP_END=0x40000000 が compile/Makefile:90 にハードコードされ、しかも 0x40000000-0x7FFFFFFF (実 DRAM) を **Device-nGnRnE でマップ**している (system/mmu.c:221-228) 触ることすらできない。mailbox の GET_ARM_MEMORY か、既にパース済みの DTB /memory から取るべき。

3.8 (14) TCP に再送も分割もない

- **再送なし**。struct tcp_conn に RTO タイマもタイマフィールドもない。
- **ウィンドウなし**。送出ウィンドウは 0x2000 固定、相手の広告ウィンドウは読まない。
- **順序外の再組み立てなし**。seq == peer_seq のときだけ追記し、他は黙って捨てる。
- **分割なし**。tcp_send() は 1 フレーム超を拒否するのに、tcp_app_flush() は最大 16384 バイトを 1 回で渡す。**約 1464 B を超える HTTP 応答は丸ごと落ちるのに g_app_served++ は回り FIN も出る**ので、クライアントには「0 バイトの正常終了」に見える。
- **ISN が 0xDEADBEEF + 接続ごとに 0x100** で完全に予測可能、かつ ESTABLISHED で ACK のウィンドウ検査をしないので、経路外からの注入が容易。

3.9 (15) GENET TX が 50 ms のビジーループ

genet_tx_frame() (device/genet/genet.c:902-952) は TDMA_CONS_INDEX を最大 50 ms ポーリングする。yield しない。ARP 応答も ICMP 応答も ACK も HTTP 応答もこのループを通る。256 個ある TX ディスクリプタのうち、常に 1 個しか使っていない。しかも送信バッファは tx_buf_pool 1 枚を全呼び出し元で共有し、各呼び出し元も自前の tx_frame[1518] を持つので**パケットごとに 2 回コピー**している。

加えて genet_rx_poll() は**ディスクリプタのエラー／status ビットも DMA_OWN も SOP/EOP も検査せず**、length (12 ビット、最大 4095) を RX_BUF_LENGTH(2048) にクランプもしない。CMD_PROMISC が常時 ON で、フィルタはソフトウェアの IP 比較のみ。

3.10 (16) DHCP が飾り

dhcp_client.c:340-352 は BOUND まで到達してリースをログに出すが、tcp_set_ip() の呼び出し元がゼロで、net_responder.c には IP セッターすら存在しない。IP は 192.168.3.100 のハードコード。T1/T2 更新もリース失効も DECLINE/RELEASE もない。

3.11 (17) ユーザ空間が存在しない

全てが EL1 の単一アドレス空間で動く。TCR.EPD1=1 で TTBR1 は無効 (system/mmu.c:240)、ASID なし、SP_EL0 不使用、**svc 命令はツリー内に 1 つもない**。シェルは 51 個の C 関数ポインタの線形探索 (shell/shell.c:926-978) で、ハンドラはカーネル内部を直接呼ぶ。cc/ が JIT したコードも EL1 特権のまま実行可能ヒープ上で走る。

Tier 0 の (1)(5)(6)(12) が全て「EL0 がないことの緩和策」である以上、いずれ避けられない分岐点である。ただし /mmio-write や /chainload が**無認証で任意ハードウェア書き込み・任意コードロード**を提供している現状では、まずネットワーク面を締めるほうが先である。

4 Tier 2 — ファイルシステムとエンジニアリング実践

4.1 (18) FAT32 にキャッシュがなく、共有 static が無防備

- fat32_next_cluster() (fs/fat32.c:96-107) は 1 エントリごとに 512 バイト読む。N クラスターのファイルを辿るのに FAT 読みだけで N 回の SD アクセス。
- fat_find_free() (:290-302) はクラスター 1 個試すごとに 1 セクタ読む → 書き込みは $O(\text{クラスター数}^2)$ 。
- static unsigned char scratch[512] (:19) を 5 つの関数が共有。プリエンティブなスケジューラの下で sdmount プロセス・kexec_tick・AVM ストリーミングから到達しうるが、**ロックは一切ない**。
- **一貫性の保証がない**。fat32_write_file_full は新クラスターを確保する前に旧チェーンを解放するので、途中で電源が落ちるとディレクトリエントリが解放済みクラスターを指す。FSInfo も更新しないので、Linux/macOS でマウントすると空き容量が狂う。
- 削除・mkdir・truncate・rename・LFN なし。ルートディレクトリのみ (パス解決がない)。

4.2 (19) VFS が抽象化になっていない

include/vfs.h:22-32 は単一の具象 vfs_node_t。file_ops の関数ポインタもマウントテーブルも inode もファイルディスクリプタも open/close/seek もない。vfs_write は**オフセット引数を持たずファイル全体**を置き換える。FAT32 は完全に別 API で、VFS には配管されていない。

4.3 (20) 自動テストが実質ゼロ

これが最大の構造的ギャップである。31k 行・195 コミットに対して:

- make qemu-smoke は出力を tee するだけで**何もアサートしない**。しかも現在リンクエラーでビルドすら通らない (sd_last_int / xhci_msd_* が未定義。ccb6437 時点で既に破綻)。
- examples_http/test.sh は 192.168.3.100 固定の curl 7 発、期待値比較なし。
- CI なし。panic() も assert() もない。-Wall のみ。

fs/fat32.c は read/write コールバックを受け取るので、FAT32 イメージファイルに対する**ホスト側テスト**が新しい抽象化なしで今すぐ書ける。fs/vfs.c と cc/ も同様。ここが投資効率の最も高い領域である。

4.4 (21) 重複と腐敗

- str_eq/puts_dec/puts_hex の near-duplicate が約 21 箇所。
- アクターランタイムが 3 実装並存 (system/actor.c / system/actorproc.c / avm.c 内蔵)。
- 陳腐化したコメント: fat32.h:1 「read-only FAT32 reader」(write 実装済み)、proc.h:12 「No pre-emption yet」(実装済み)、shell.c:966 「no scheduler yet」。
- デッドコード: device/usb/dwc2.c、include/qemu_repro_src.h、repro コマンド、ビルドルールから参照されない examples/ の 11 ファイル。
- **make install_pi4 は地雷**。config.txt を上書きして total_mem=2048 を落とす。この事実は NEXT_SESSION_PI4.md にしか書かれておらず、Makefile 自身は警告しない。

5 推奨着手順序と進捗

順	内容	効果	工数	状態
1	(1)(2) ヒープ分割 + irq_save	クリティカル	30 分	✓
2	(3)(4) パケット長検証 + half-open reap	情報漏洩 / DoS	半日	✓
3	(5) node->data NULL ガード	サイレント破壊の代 表	半日	✓*
4	(7) 優先度プリエンプション修正	RTOS の看板そのも の	1 時間	✓
5	(8) ready キュー O(1) + sleep 整列リスト	アクター系の実測が 動く	1-2 日	✓
6	(12) スタックカナリア	サイレント破壊の封 じ込め	半日	✓
7	(14) TCP 分割・再送・ウィンドウ	HTTP が 1.4KB で 壊れない	1 日	✓
8	ホストテスト基盤 + CI	以降の全変更の安全 網	半日	一部
9	(6) W^X 修正	.data/.bss が RWX	半日	—
10	(10) genet/xhci の DMA 規律 → DCACHE_ON	最大の性能改善	2-3 日	—
11	(15) GENET TX リング、50ms スピン除去	スループット / レイ テンシ	1-2 日	—
12	(9) タイムベース、(13)(16) 2GB 開放・DHCP 適用、(11) network/ の去就	中期	数日	—
13	(17) EL0/ユーザ空間分離、SVC 境界	OS としての次の段 階	大	—

* /microsd が空でマウント未成立のため実機未検証。

6 第 1 次改良 — Tier 0 の 5 件

コミット e6973d5。ビルド md5 74d3ff7f (修正前 cfeeb18c)。

#	修正	ファイル
1	MEM_ALIGN を 8 → 16 (= sizeof(struct memblk))。分割の余りは 0 か 16 以上にしかならず、getmem と freemem 双方のはみ出しが 構造的に消える 。副次的に AArch64 が要求する 16 バイトスタックアラインメントも満たす	include/memory.h mem/memory.c
2	ヒープ操作を irq_save/irq_restore で保護	mem/memory.c
3	ICMP と TCP の長さ検証。total_len・data_off を実受信長と照合	net_responder.c tcp_server.c
4	アイドル接続リーパー。half-open 5 秒 / ESTABLISHED 60 秒でスロット回収	tcp_server.c loader/main.c
5	cat/cp/mv の node->data NULL ガード	shell/fscmd.c

6.1 ホストテスト基盤の追加 — test/host/

```
make -C test/host run
```

カーネルの純ロジックをビルドマシン上で直接走らせる枠組み。mem/memory.c をそのまま#include し、critical.h だけホスト用スタブで差し替える。インクルード順で実カーネルヘッダを使うので、テストと本番が同じ定義を共有する。

この枠組みは実際に仕事をした。目視では (1) の getmem 側しか見つけておらず、freemem 側の同型バグは 20000 回のランダム churn テストが検出した。歯があることの確認として、修正前のツリーに同じテストを掛けると **5 件 FAIL**、修正後は全 PASS になる。

6.2 実機検証結果

項目	結果
ICMP (リグレーション確認)	ping 5/5 応答、パケットロス 0%
HTTP	GET /, /ticks 正常。txfail=0
リーパーの誤検出	正常接続 10 本で reaped=0、drop_syn=0
リーパーの DoS 解除	アイドル接続でスロット占有 → reaped=3 、全スロット LISTEN に復帰
ヒープ	free 977733 KiB、 free blocks = 1 (完全合体)
cat ガード	未検証 (/microsd が空でマウント未成立)

7 第 2 次改良 — Tier 1 の (7)(8)(12)(14)

コミット `ce106cc` (スケジューラ) と `256dbe9` (ネットワーク)。ビルド `md5 becaf060`。実機検証済み。

7.1 スケジューラ

修正	実測
(7) 走行中プロセスを <code>pop</code> の前に <code>requeue</code> し、自分の優先度を比較に参加させる。同優先度のラウンドロビンも自然に得られる	<code>/rtos-jitter 10ms</code> 周期・CPU hog 下で max jitter 10μs / ミス 0/100 (協調モード: 5162 μ s / 46 ミス)、プリエンブション 1002 回発火
(8) 優先度別 FIFO + 64bit ビットマップ (CLZ 1 発)、 <code>sleep</code> は deadline 整列リスト	SMP ベンチ劣化なし (dining 3.99 $\times$ 、primes 3.01 $\times$ 、n-queens 1.80 $\times$ 、agree=yes)
(12) <code>stkbases</code> にカナリア、全スイッチ点で検査、 <code>/ticks</code> に <code>stkbases=</code>	全負荷試験を通して <code>stkbases=0</code>

作業中に見つけた既存バグも同時に修正した:

- `shell.c` の `procdemo` が `ready` キューから外さずに `PR_FREE` を直書き (次のディスパッチが解放済みスタックへ `ctxsw` する)。
- `proc_ready()` が `PR_FREE/PR_TERM` を受け付ける。
- `proc_ready()` が `PR_SLEEP` を `sleep` リストから外さずに `ready` へ入れる (両リストが同じ `next` を共有)。
- `proc_block/proc_sleep_us` が `NULLPROC` を自分の古い SP へ `ctxsw` しうる。

`test/host/proctest.c` を追加 (14 テスト)。 `ctxsw` をスタブ化して実物の `proc.c` を走らせ、毎回ビットマップ/FIFO/カウント/sleep リストの不変条件を検査する。核心のテストは `ccb6437` の順序に対して FAIL する — そこでは `prio-50` のタスクが `prio-5` の hog に叩き落とされ、実機と同じ挙動になる。

7.2 ネットワーク (14)

- **分割:** コネクションごとの送信バッファ + MSS 単位の `tcp_pump()`。FIN は全バイト ACK 後。実測 `GET /shell?cmd=help` が **2618 バイト = Content-Length 2618** (従来は 0 バイト)、6 回連続で同一 `md5`、`retrans=0`。
- **再送:** RTO 500ms の go-back-N、6 回で打ち切り。相手の広告ウィンドウを尊重。net プロセスは RX 割り込みでしか起きないため、wm ループが `tcp_needs_tick()` で必要時のみ起こす (送信は `tx_frame` 共有のため net プロセスに一本化)。
- **http_build の max 無視:** `body[16384]` + ヘッダを `out[16384]` に書いて約 120 バイト BSS を溢れさせていた。

7.3 /chainload が箱を落としていた真因

ステージングアドレス **0x4000000** が、走行中カーネル自身の **.bss** の内側にあった。

ステージング先	0x4000000 = 67,108,864
カーネル _end	0x4233B18 = 69,286,168 <- 2.08 MB 上

アップロードの全域が生きたカーネル状態を上書きするため、go=1 を撃つ前、**転送中に必ず死ぬ**。本セッション中に 2 回落とした。チェーンロード実装当時カーネルは約 115 KB で、.bss が 69 MB に育つ過程で固定アドレスが飲み込まれた**設計の腐食**である。README とマニュアルはこれを「SD 交換不要の安全な高速反復パス」と謳い続けていた。

修正: ヒープから確保 (必ず _end より上)、オフセットの境界検査、go=1 時に走行中イメージと重なるステージングを拒否。system/kexec.c:108 が既にヒープバッファを kernel_chainload() に渡しており、実績のあるパターンだった。

実測: 2,008,152 バイトを 23.1 秒で転送、全 123 チャンク ACK、ステージしたカーネルが起動。これで実機検証が SD カードの往復なしで回せるようになった。

7.4 決着した懸案 — /preempt は既存の壊れ

/preempt が空ログを返し cpuA/cpuB が居残る件は、修理したチェーンロードで旧カーネル (ccb6437) を RAM 起動して A/B した結果、**旧カーネルでも同じと確認できた** (ログは BA / B / AB / 空と毎回変動、期待値 20 文字。子プロセスは呼ぶたび 2 個ずつ蓄積)。スケジューラ変更による回帰ではない。

原因は preempt_demo 自身のコメントが書いている前提 — 「shell/NULLPROC コンテキストから実行する」。HTTP 経由だと app ワーカー (prio 1) が呼び出し元になり、子と同格なのでラウンドロビンで先に戻ってしまう。NULLPROC は ready キューに載らないのでシリアルシェルからなら意図どおり動くはず。procdemo が途中で戻るのと同じ構図 (こちらはホスト側で新旧のディスパッチ順が同一と実証済み)。

8 未解決の課題

- /microsd と /sd が空。SD の FAT32 列挙が成立していない。(5) の検証を阻んでおり、FAT32 経路が実機で一切テストできない。**次の本命**。
- make qemu / qemu-smoke がリンクしない (sd_last_int、xhci_msd_* が未定義)。ccb6437 時点で既に破綻していた。
- 残る優先課題は表の 9-13 — W^X、DMA キャッシュ規律 → DCACHE_ON、GENET TX リング、そして EL0 分離。

本書は docs/OS_ASSESSMENT_JA.md の LaTeX 清書版である。数値はすべて実機 (Raspberry Pi 4 / 192.168.3.100) での測定値。